

Course Outline: Introduction to AI Agents

Title: Introduction to AI Agents **Learnpack:** + Introducción a agentes **Prerequisite:** Skill 19 — Building a Python API to Serve the Frontend (Week 8, Day 22) **Target Audience:** Beginner developers who have built APIs with FastAPI and understand HTTP request-response flows **Total Lessons:** 12 **Estimated Total Length:** ~6,000 words **Format:** Conceptual (0% hands-on)

Course Description

This course introduces students to the concept of AI agents — systems that go beyond simple question-and-answer interactions with a language model. Students will learn what distinguishes an agent from a plain LLM call, how agents operate through a recurring cycle of observation, decision, and action, and what components are needed to design one.

The course builds on students' existing knowledge of APIs and LLMs (covered in the prework) to frame agents as a natural evolution: instead of a single request-response, the system loops, reasons, and acts until a goal is met. This purely conceptual foundation prepares students for the next learnpack, where they will build a working agent in Python.

Course Philosophy

This course emphasizes:

- **Thinking in cycles, not single responses:** An agent is not a smarter prompt — it is a loop with a purpose.
 - **Components before code:** Understanding Objective, Model, Memory, Tools, and Loop conceptually before writing any implementation.
 - **Recognizing when NOT to use an agent:** Not every problem needs an autonomous loop; knowing the boundary matters.
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 — Welcome	1	~450
01 — What Is an Agent?	3	~1,500
02 — The Agent Loop	3	~1,500
03 — Agent Components	3	~1,500
04 — Assessment	1	~700
05 — What's Next	1	~450
Total	12	~6,100

Section 00: Welcome

00.0 Welcome 📖 (~450 words)

Learning Objectives:

- Identify what this course covers and what it does not
- Connect the concept of agents to what students already know about LLMs and APIs

Content Outline:

- Brief recap: students already know how to call an LLM (prework) and build APIs (Skill 19)
- The question this course answers: what happens when the LLM needs to do more than just reply?
- Course roadmap: what an agent is, how it works, what it is made of, and common design mistakes
- Clarification: this course is conceptual — students will build a real agent in the next learnpack

Transitions: This lesson sets the stage. The next section dives into the core distinction between an LLM and an agent.

Key Principles:

- An agent is not a smarter prompt — it is a system with a goal, a loop, and the ability to act
- Understanding the concept first makes implementation faster and less error-prone

Examples:

- A chatbot that answers questions (LLM) vs. a system that books a flight by searching, comparing, and confirming (agent)
 - A student asking Rigobot a question (single call) vs. Rigobot reviewing an entire codebase and suggesting fixes across files (agent-like behavior)
-

Section 01: What Is an Agent?

01.0 LLMs vs Agents 📖 (~500 words)

Learning Objectives:

- Differentiate between a plain LLM call and an AI agent
- Explain why a single input-output interaction is not enough for certain tasks

Content Outline:

- The mental model most people have: input → LLM → response (one shot)
- What an agent does differently: input → loop → decision → action → new information → loop
- Key distinction: an LLM generates text, an agent uses that text generation to reason and then acts on the world
- The agent keeps going until the goal is met or a stopping condition is reached

Transitions: Now that students understand the distinction, the next lesson explores the internal mechanics of how an agent operates.

Key Principles:

- An LLM is a component inside an agent, not the agent itself
- The defining feature of an agent is the loop: it does not stop after one response

Examples:

- Asking ChatGPT "What is the weather?" (LLM: generates a text answer, cannot actually check weather)
 - An agent that receives "Book me a hotel in Madrid": it searches availability, compares prices, selects the best option, and confirms the reservation — multiple steps, multiple actions
 - A customer support bot that only answers FAQs (LLM) vs. one that looks up the user's order, checks shipping status, and issues a refund if needed (agent)
-

01.1 How an Agent Works 📖 (~500 words)

Learning Objectives:

- Describe the general execution flow of an agent at a high level
- Identify the repeating nature of the agent cycle

Content Outline:

- The simplified flow: Goal → Observe → Reason → Act → Update → Repeat
- Each iteration: the agent receives new information, decides what to do next, executes an action, and feeds the result back into its context
- The loop ends when the goal is achieved, a maximum number of iterations is reached, or an error condition triggers a stop
- Pseudocode sketch of the cycle (conceptual, not implementation)

Transitions: Students now understand the general flow. The next lesson challenges them to distinguish between systems that are agents and systems that are not.

Key Principles:

- The agent loop is the core mechanism — without it, there is no agent
- Every iteration changes the agent's context, which influences the next decision

Examples:

- Pseudocode: `while not done: observation = observe(); decision = reason(observation); result = act(decision); update(result)`
 - A research agent: reads a question → searches the web → reads the top result → decides it needs more info → searches again → synthesizes an answer → done
 - Comparison with a for loop in programming: the agent loop is conceptually similar but the "condition" is determined by the LLM's reasoning
-

01.2 Not Everything Needs an Agent 📖 (~500 words)

Learning Objectives:

- Identify situations where a simple LLM call is sufficient and an agent is unnecessary
- Recognize the cost and complexity an agent adds compared to a direct API call

Content Outline:

- Anti-pattern: treating every LLM interaction as if it needs an agent
- When a single prompt is enough: translation, summarization, classification, simple Q&A
- When an agent is warranted: multi-step tasks, tasks requiring external data retrieval, tasks with conditional logic that depends on intermediate results
- The cost of unnecessary agents: more tokens consumed, more latency, harder to debug, higher chance of failure
- Rule of thumb: if the task can be completed in one LLM call with all necessary information in the prompt, do not build an agent

Transitions: With a clear understanding of what agents are (and are not), the next section dives deeper into the agent loop — the engine that drives every agent.

Key Principles:

- More complexity is not better — the path of least resistance applies to agent design too
- An agent should only exist when the task genuinely requires multiple steps with intermediate decisions

Examples:

- Anti-pattern: building an agent to translate a paragraph from English to Spanish (one LLM call is enough)
 - Anti-pattern: building an agent to answer "What is Python?" (a direct response suffices)
 - Valid use: an agent that monitors a GitHub repository, checks for new issues, classifies them by priority, and assigns them to team members — this genuinely requires multiple steps and tool use
-

Section 02: The Agent Loop

02.0 Observe, Decide, Act 📖 (~500 words)

Learning Objectives:

- Break down the agent loop into its three core phases: Observe, Decide, Act
- Explain what happens in each phase

Content Outline:

- **Observe:** The agent gathers information — this could be the user's initial input, the result of a previous action, or data from an external source
- **Decide:** The LLM reasons about the current state and determines the next step — should it call a tool, ask for clarification, or return a final answer?
- **Act:** The agent executes the decision — calling a function, querying an API, writing to a file, or returning a response to the user
- After acting, the result feeds back into the Observe phase, and the cycle repeats
- The pattern name: Observe → Decide → Act (from the syllabus thinking framework)

Transitions: The next lesson expands on what "thinking in cycles" really means and how it differs from the linear thinking students are used to.

Key Principles:

- Each phase has a distinct responsibility: gathering data, reasoning, and executing
- The LLM handles the Decide phase; code handles the Observe and Act phases

Examples:

- An email-sorting agent: Observe (read the next email), Decide (is this spam, important, or routine?), Act (move to the corresponding folder). Repeat for the next email.
 - A coding agent: Observe (read the error message), Decide (the issue is a missing import), Act (add the import statement). Observe again (run tests), Decide (tests pass), Act (report success to user).
-

02.1 Thinking in Cycles 📖 (~500 words)

Learning Objectives:

- Shift from linear "one request, one response" thinking to cyclical "loop until done" thinking
- Identify the stopping conditions that end an agent loop

Content Outline:

- The mental shift: programming students are used to functions that run once and return — agents run repeatedly until a condition is met
- Stopping conditions: goal achieved, maximum iterations reached, unrecoverable error, user cancellation
- Why maximum iterations matter: without a limit, an agent can loop forever (wasting tokens and time)
- The analogy: an agent is like a while loop where the condition is evaluated by the LLM's reasoning, not by a simple boolean

Transitions: Students now understand the loop. The next lesson presents a common anti-pattern: designing loops without clear stopping conditions.

Key Principles:

- Every agent loop must have a defined exit strategy
- The LLM decides when the goal is met, but the code enforces hard limits (max iterations)

Examples:

- A shopping agent that searches, compares, and buys: it stops when the purchase is confirmed or after 10 iterations (safety limit)
 - A data-cleaning agent that processes rows: it stops when all rows are processed or when it encounters an error it cannot resolve
 - The danger: an agent asked to "find the perfect restaurant" with no stopping criteria could search forever
-

02.2 The Infinite Loop Trap 📖 (~500 words)

Learning Objectives:

- Recognize the anti-pattern of designing agent loops without clear termination conditions
- Identify the symptoms and consequences of a runaway agent loop

Content Outline:

- Anti-pattern: not defining a condition of finalization (from syllabus anti-patterns)
- What happens: the agent keeps iterating — consuming tokens, making API calls, and never returning a result
- Common causes: vague objectives ("make it perfect"), no maximum iteration count, the agent's reasoning enters a circular pattern (tries the same action repeatedly)
- Anti-pattern: prompt too complex — when the agent's instructions are overloaded, it struggles to determine when it is "done"
- How to prevent it: always set a max iteration limit, define success criteria explicitly, log each iteration so you can detect circular behavior

Transitions: With the loop understood (including its risks), the next section breaks down the five components every agent needs.

Key Principles:

- A loop without an exit is not an agent — it is a bug
- Simplicity in the agent's objective directly reduces the risk of infinite loops

Examples:

- Bad: "Research everything about renewable energy and compile a comprehensive report" (no clear "done" signal)
 - Good: "Find the top 3 solar panel brands by efficiency rating and return a comparison table" (clear deliverable, clear stopping point)
 - A student's agent that keeps calling a weather API every iteration because the prompt says "make sure the data is accurate" — it never stops verifying
-

Section 03: Agent Components

03.0 Objective and Model 📖 (~500 words)

Learning Objectives:

- Define the role of the Objective in guiding an agent's behavior
- Explain how the Model (LLM) serves as the agent's reasoning engine

Content Outline:

- **Objective:** The purpose the agent exists to fulfill — without a clear objective, the agent has no direction
- A good objective is specific, achievable, and has a measurable end state
- **Model (LLM):** The reasoning core — the agent sends observations and context to the LLM, and the LLM returns a decision

- The model does not execute anything — it only reasons and produces text (the code layer handles execution)
- Choosing a model: cost, speed, and capability tradeoffs (reference to prework Skill 0.5 on choosing between LLMs)
- Pseudocode showing how the objective is passed to the model as part of the system prompt

Transitions: With Objective and Model covered, the next lesson introduces the remaining three components: Memory, Tools, and the Loop itself.

Key Principles:

- The Objective is the "why" — without it, the agent is aimless
- The Model is the "brain" — it reasons but does not act directly
- LLM is reasoning, Code is execution (from syllabus thinking framework)

Examples:

- Objective: "Given a user's question about our product, find the relevant documentation page and return a summary with a link"
 - The model receives: system prompt (objective + instructions) + current context (user question + previous search results) → returns: "I should search the docs for 'pricing plans'"
 - Bad objective: "Help the user" (too vague — the agent cannot determine when it is done)
-

03.1 Memory, Tools, and Loop 📖 (~500 words)

Learning Objectives:

- Explain the role of Memory/Context in maintaining state across iterations
- Define what Tools are and how the agent selects them
- Describe how the Agent Loop ties all components together

Content Outline:

- **Memory / Context:** Everything the agent "remembers" between iterations — the original input, previous actions, results, and accumulated knowledge. Without memory, each iteration starts from scratch.
- **Tools:** Functions the agent can call to interact with the outside world — search the web, query a database, send an email, call an API. The agent does not hardcode which tool to use; it decides based on the current observation.
- Tool selection pattern: the agent's reasoning determines which tool fits the situation (from syllabus patterns)
- Result injection pattern: after a tool executes, its output is added to the agent's context for the next iteration (from syllabus patterns)
- **Agent Loop:** The orchestrator — it runs the Observe → Decide → Act cycle, manages memory, invokes tools, and checks stopping conditions
- Pseudocode tying all five components together: objective, model, memory, tools, loop

Transitions: Students now know all five components. The next lesson presents common mistakes when designing these components.

Key Principles:

- Memory is what makes each iteration smarter than the last
- Tools are the agent's hands — the LLM reasons, tools execute
- The loop is the glue that connects everything

Examples:

- Memory: after searching for "hotels in Madrid," the agent remembers it found 3 options — it does not search again
- Tools: `search_web(query)`, `get_weather(city)`, `send_email(to, subject, body)` — each is a simple, specific function
- Complete pseudocode:

```
memory = [user_input]
for i in range(max_iterations):
    decision = model.reason(objective, memory)
    if decision == "done": return memory.final_answer
    result = tools.execute(decision.tool, decision.args)
    memory.append(result)
```

03.2 Too Many Tools, No Clear Goal 📖 (~500 words)

Learning Objectives:

- Recognize the anti-patterns of overloading an agent with tools and using ambiguous tool descriptions
- Apply the guideline of keeping tools simple, specific, and limited in number

Content Outline:

- Anti-pattern: too many tools (from syllabus) — when an agent has 15 tools available, the LLM struggles to pick the right one, wastes tokens evaluating options, and often chooses incorrectly
- Best practice: 2-4 tools per agent (from syllabus best practices)
- Anti-pattern: ambiguous tools (from syllabus) — if two tools have overlapping descriptions, the agent cannot reliably distinguish between them
- Anti-pattern: putting too much logic inside a tool — a tool should do one thing. If a tool searches, filters, sorts, and formats, it is too complex; the agent should handle the orchestration
- Best practice: tools should be deterministic — given the same input, they produce the same output (from syllabus best practices)
- Anti-pattern: no clear goal — without a well-defined objective, even the best tools are useless because the agent does not know what "done" looks like

Transitions: Students have now learned what agents are, how they work, what they are made of, and what mistakes to avoid. The next section tests this understanding.

Key Principles:

- Fewer, clearer tools lead to better agent behavior

- A tool should be simple enough that its description fits in one sentence
- The objective must define "done" — otherwise the agent cannot stop

Examples:

- Bad: an agent with tools for `search_web`, `search_docs`, `search_faq`, `search_knowledge_base`, `search_wiki` — the LLM cannot distinguish them
 - Good: one `search(source, query)` tool with a source parameter, or two clearly distinct tools: `search_public_web(query)` and `search_internal_docs(query)`
 - Bad tool: `process_order(order_id)` that validates, charges, sends confirmation, and updates inventory — this should be 4 separate tools
 - Good tool: `charge_payment(order_id, amount)` — one action, one result
-

Section 04: Assessment

04.0 Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of the difference between LLMs and agents
- Identify agent components and their roles
- Recognize anti-patterns in agent design

Question Format: Scenario-based (all questions follow this format)

Questions:

Question 1: A developer builds a system that takes a user's question, sends it to an LLM, and returns the response directly. The user asks: "What flights are available from Bogotá to Madrid next week?" The system responds with a generic paragraph about how to search for flights. Is this system an agent? Why or why not?

Expected reasoning: This is not an agent — it is a single LLM call with no tools, no loop, and no ability to actually search for flights. An agent would search a flight API, compare results, and return real data.

Question 2: An agent is designed to help users find recipes. It has 12 tools: `search_recipes`, `search_ingredients`, `search_nutritional_info`, `search_reviews`, `search_cooking_videos`, `search_chef_profiles`, `search_restaurants`, `search_meal_plans`, `search_grocery_stores`, `search_kitchen_equipment`, `search_cooking_classes`, `search_food_blogs`. What problem will this agent likely have?

Expected reasoning: Too many tools — the LLM will struggle to choose the right one. The agent should have 2-4 focused tools instead.

Question 3: An agent is given the objective: "Make the user happy." It has tools to search the web, send emails, and check the calendar. After 50 iterations, it is still running. What is the most likely cause?

Expected reasoning: The objective is too vague — "make the user happy" has no measurable end state, so the agent cannot determine when it is done. Combined with no iteration limit (or a limit set too high), the agent loops indefinitely.

Question 4: A developer describes their agent's flow: "The user sends a message, the LLM generates a response, and the response is returned." They claim this is an agent because the LLM is "intelligent." Is this an agent? What is missing?

Expected reasoning: This is not an agent — it is a standard LLM call. Missing components: no loop (it runs once), no tools (it cannot act on the world), no memory across iterations, and arguably no explicit objective beyond "respond."

Question 5: An agent has a tool called `do_everything(task_description)` that takes a free-text description and internally handles searching, processing, and returning results. What anti-pattern does this represent?

Expected reasoning: Too much logic inside a single tool. The tool is not specific or deterministic — it is a black box. The agent should have separate, simple tools for each action so the LLM can orchestrate them.

Question 6: In the Observe → Decide → Act cycle, which component is responsible for the "Decide" phase, and which handles "Act"?

Expected reasoning: The LLM (Model) handles the Decide phase — it reasons about what to do next. The code layer (Tools) handles the Act phase — it executes the chosen action. The LLM generates text decisions; code translates those into real actions.

Evaluation Criteria:

- Students should demonstrate understanding of the conceptual distinction between LLMs and agents
- Students should correctly identify the five agent components and their roles
- Students should recognize anti-patterns (too many tools, vague objectives, no stopping condition, too much logic in tools)

Score Interpretation:

- 6/6: Strong conceptual foundation — ready to build an agent in the next learnpack
- 4-5/6: Good understanding with minor gaps — review the missed concepts before proceeding
- Below 4/6: Revisit the course material, focusing on the agent loop and component roles

Section 05: What's Next

05.0 From Concepts to Code (~450 words)

Content Outline:

- Recap: students now understand what an agent is (a system that loops, reasons, and acts), how it differs from a plain LLM call, the five components (Objective, Model, Memory, Tools, Loop), the Observe → Decide → Act cycle, and the most common design mistakes
- What comes next: in the next learnpack ("Creación de un agente de IA básico en Python"), students will build a working agent from scratch — implementing the loop, defining tools, writing agent prompts, and handling real execution
- The conceptual understanding from this course will directly inform implementation decisions: how many tools to create, how to define the objective in the system prompt, how to structure memory, and when to stop the loop

- Encouragement: agents are one of the most exciting areas in AI engineering right now — students are building skills that are in high demand
- Reminder: the best agents are simple agents with clear goals, few tools, and well-defined stopping conditions

Note: This is conclusion only — no theory, exercises, or assessment.